

Linux and Command-Line Fundamentals - Study Notes

1. The Filesystem The Linux filesystem is a single tree that starts at the root directory, written as `/`. There is no drive letter like `C:` in Windows. Key directories: `/home` holds user home directories, `/etc` holds system configuration files, `/var` holds variable data such as logs, `/bin` and `/usr/bin` hold executable programs, `/tmp` holds temporary files that are usually cleared on reboot, and `/dev` holds device files. An absolute path starts with `/`, such as `/home/ryan/notes.txt`. A relative path is interpreted from the current working directory. The shortcut `.` means the current directory, `..` means the parent directory, and `~` means the current user's home directory. Any file or directory whose name begins with a dot, such as `.bashrc`, is hidden from a plain `ls` listing; use `ls -a` to see hidden files.

2. Navigating and Inspecting `pwd` prints the current working directory. `cd` changes directory, and `cd` with no arguments returns you to your home directory. `ls` lists directory contents: `ls -l` gives the long listing with permissions, owner, size and modification time; `ls -a` includes hidden files; `ls -lh` with `-l` prints human-readable sizes such as `4.0K` or `2.1M`. `cat` prints an entire file to the screen. `less` opens a file in a scrollable pager, and you quit `less` by pressing `q`. `head` prints the first 10 lines of a file by default, and `tail` prints the last 10 lines. `tail -f` follows a file as it grows, which is the standard way to watch a log file in real time. `wc -l` counts lines, and `file` reports what kind of data a file contains.

3. Permissions Every file has an owner, a group, and three sets of permissions: user (the owner), group, and other. The three permission bits are read (`r`), write (`w`), and execute (`x`). In an `ls -l` listing the first character is the file type: `-` for a regular file, `d` for a directory, and `l` for a symbolic link. The next nine characters are the three permission triples, so `-rwxr-xr--` means the owner can read, write and execute, the group can read and execute, and others can only read. Permissions are often written in octal, where read is 4, write is 2 and execute is 1, and the three digits are added per triple. So `755` means `rwxr-xr-x` and `644` means `rw-r--r--`. `chmod` changes permissions, for example `chmod 755 script.sh` or `chmod +x script.sh` to make a file executable. `chown` changes the owner of a file, for example `chown ryan:ryan file.txt`. On a directory, the execute bit means permission to enter the directory rather than permission to run it. `sudo` runs a single command as another user, by default root, and is preferred over logging in as root directly because it leaves an audit trail.

4. Redirection and Pipes Every process starts with three standard streams: standard input (stdin, file descriptor 0), standard output (stdout, file descriptor 1), and standard error (stderr, file descriptor 2). The `>` operator redirects stdout to a file and overwrites that file. The `>>` operator redirects stdout to a file and appends to it instead. The `<` operator redirects a file into a command's stdin. `2>` redirects stderr, and `2>&1` sends stderr to the same place stdout is already going. The pipe operator `|` connects the stdout of one command to the stdin of the next, so commands can be chained: `ls -l | grep ".txt" | wc -l` counts how many entries in the long listing contain `.txt`. `/dev/null` is a special file that discards everything written to it, so `2>/dev/null` throws away error messages.

5. Finding Things `grep` searches file contents for a pattern. `grep -i` ignores case, `grep -r` searches recursively through directories, `grep -n` shows line numbers, and `grep -v` inverts the match to show lines that do NOT match. `find` searches the filesystem by attributes rather than contents: `find . -name "*.log"` finds files by name, `find . -type d` finds directories, and `find . -mtime -1` finds files modified in the last day. `which` shows the full path of the executable that would run for a given command name. `man` opens the manual page for a command.

6. Processes A process is a running program, identified by a numeric process ID, or PID. `ps aux` lists all running processes with their owners and resource use. `top` and `htop` show a live, continuously updating view of processes sorted by CPU use. `kill` sends a signal to a process by PID. By default `kill` sends `SIGTERM`, signal 15, which politely asks the process to shut down and clean up. `kill -9` sends `SIGKILL`, signal 9, which the process cannot catch or ignore and which terminates it immediately without cleanup, so `SIGTERM` should always be tried first. Adding `&` to the end of a command runs it in the background. `jobs`

lists background jobs of the current shell, `fg` brings a job to the foreground, and `Ctrl+C` sends an interrupt that stops the process running in the foreground. `Ctrl+Z` suspends it instead.

7. The Shell Environment The shell is the program that reads and interprets your commands; `bash` and `zsh` are the two most common. Environment variables carry configuration into programs, and they are conventionally uppercase. `echo $HOME` prints your home directory, `echo $PATH` prints the colon-separated list of directories the shell searches for executables, and `env` lists all environment variables. `export VAR=value` sets a variable so that child processes inherit it. Variables set without `export` exist only in the current shell. Shell startup files such as `~/.bashrc` run each time an interactive shell starts, which is where aliases and `PATH` changes are usually put. An alias is a short name for a longer command, for example `alias ll='ls -lah'`. Tab completion completes file and command names, and the up arrow recalls previous commands from the shell history, which `history` prints.

8. Files and Archives `touch` creates an empty file or updates a file's timestamp. `mkdir` creates a directory, and `mkdir -p` creates any missing parent directories along the way. `cp` copies files, and `cp -r` copies a directory recursively. `mv` both moves and renames files, since renaming is just moving within the same directory. `rm` deletes files and `rm -r` deletes directories recursively; there is no recycle bin, so deletion is immediate and permanent. `rmdir` removes only an empty directory. `tar` bundles many files into one archive. `tar -czf backup.tar.gz mydir/` creates a gzip-compressed archive, and `tar -xzf backup.tar.gz` extracts it, where `c` means create, `x` means extract, `z` means use gzip, and `f` names the file. `df -h` reports free disk space per filesystem, and `du -sh` reports the total size of a directory.